

Trimming whitespaces before line splicing

Document #: D2223R0
Date: 2020-09-14
Project: Programming Language C++
Audience: EWG, CWG
Reply-to: Corentin Jabot <corentin.jabot@gmail.com>

Abstract

We propose to make trailing whitespaces after `\` non-significant.

This paper has been spawn off [P2178R0](#) [1] and is part of a larger effort from SG-16 to improve lexing.

Making trailing whitespaces non-significant

There is a divergence of implementation in how compilers handle spaces between a backslash and the end of a line.

```
int main() {  
    int i = 1  
    // \  
    + 42  
    ;  
    return i;  
}
```

EDG(tested with icc front-end) GCC and Clang will trim the whitespaces after the backslash - and return 1 - MSVC will not and return 43. Both strategies are valid as part of phase 1 "implementation-defined mapping". There is a clang issue about this [#15509](#)

To avoid this surprising implementation divergence we proposed that an implementation must trim all trailing whitespaces before handling `\` slicing. This is reinforced by the fact that IDEs and tools (code formatter for example) may discard such whitespaces. The Google-style guidelines forbid trailing whitespaces.

An additional or alternative approach is to deprecate `\` that are not part of a preprocessor directive. We are not proposing this at this time.

This proposal may be a silent breaking change for code that is only compiled with MSVC. A quick analysis of the packages available in VCPKG didn't find any trailing whitespaces after backslashes.

We have not been able to measure the impact of this proposed change in the MSVC ecosystem. Other compilers, and all code supported by these compilers would be unaffected. However, neither tools nor people can be expected to reliably preserve invisible character. The lack of usefulness and the brittleness of relying on this behavior makes it unlikely to be relied upon.

GCC, Clang, and ICC provide a warning enabled by default **warning: backslash and newline separated by space**.

String literals

Another interesting example

```
auto str "\<space>  
";
```

Is an empty string in GCC, ICC clang and the string "\ " in MSVC. As '\ ' is not a valid escape sequence, the above code is ill-formed in MSVC.

Raw string literals

The proposed trimming is intended to be reversed in raw string literals. However, the reversal of line splicing in raw strings might not be sufficiently specified, leading to some implementation divergence and open GCC issues [#91412](#), [#43606](#), which the present paper doesn't intend to address.

Whitespaces?

The proposed wording as well as the standard don't specify what characters are considered whitespaces. This will be addressed separately (see [P2178R0 \[1\]](#)), but the intent is that characters considered whitespaces for the purpose of lines splicing be the same as whitespace characters in the rest of the grammar. This includes tabs and other characters considered whitespaces by the Unicode standard.

CWG 1698

The proposed wording also offers a fix to issue [CWG1698 \[2\]](#) and specify that a backslash in the last physical source line is not considered part of a splice.

C compatibility

In effect, the "implementation defined mapping: in phase one make the content of the program implementation defined, and it is a valid behavior in C and C++ alike to trim or not trailing whitespaces. The proposed change only requires that all C++ compilers do this trimming, and therefore doesn't affect C compatibility.

Proposed Wording

Modify 5.2.1.2 as follow

❖ **Lexical conventions** [lex]

❖ **Phases of translation** [lex.phases]

Each instance of a backslash character (\) immediately followed by 0 or more whitespace characters and a new-line character is deleted, splicing physical source lines to form logical source lines. Only the last backslash on any physical source line but the last shall be eligible for being part of such a splice. [...]

References

[1] Corentin Jabot. P2178R0: Misc lexing and string handling improvements. <https://wg21.link/p2178r0>, 6 2020.

[2] David Krauss. CWG1698: Files ending in \. <https://wg21.link/cwg1698>, 6 2013.

[UAX-14] *UNICODE LINE BREAKING ALGORITHM*
<https://www.unicode.org/reports/tr14/>

[Unicode] Unicode 13
<http://www.unicode.org/versions/Unicode13.0.0/>

[N4861] Richard Smith *Working Draft, Standard for Programming Language C++*
<https://wg21.link/N4861>